

**Anwendungsszenario**

- Wir benötigen eine Datenstruktur, die das Suchen nach und Einfügen von Datensätzen unterstützt.
- Datensätze bestehen aus Schlüsselwerten zur Identifikation und mglw. zusätzlichen Informationen.
- Datensätze sollen jedoch nicht (oder nur selten) gelöscht werden.
- Zeitkritische (weil bei weitem häufigste Operation) ist das Suchen (Nachschlagen) nach Datensätzen.

Beispiel 1: Online-Wörterbuch

- Schlüsselwerte: Die eingetragenen Wörter.
- Zusätzliche Informationen: Die erklärenden Texte.
- Um das Wörterbuch aufzubauen, werden zunächst alle Einträge eingefügt.
- Benutzen des Wörterbuchs heißt dann Nachschlagen von Wörtern.
- Aus Benutzersicht ist dies zeitkritisch.

**Beispiel 4: Namenstabellen beim Compilieren**

- Wenn der Compiler ein Source-code-file parst, erstellt er eine Tabelle der verwendeten Identifier und speichert für jeden Identifier den zugehörigen Typ, wenn seine Deklaration gelesen wird.
- Wenn der Compiler beim Parsen auf einen Identifier trifft, schlägt er ihn nach und prüft somit, ob er bereits definiert wurde und mit Hilfe der Typinformation ob seine Benutzung kompatibel zu seinem Typ ist.

Im folgenden Abschnitt wollen wir ein weiteres Vorgehen kennenlernen, wie man Wörterbücher realisieren kann, sogenannte *Hashverfahren*.

Diese sind besonders für die genannten Anwendungen geeignet.

**Beispiel 2: Speicher einer WWW-Suchmaschine**

- Schlüsselwerte: Alle (wesentlichen) Wörter, die in den durchsuchten WWW-Seiten gefunden wurden.
- Zusätzliche Informationen: Zu jedem indizierten Wort, die Liste der URL's, in denen es gefunden wurde.

Beispiel 3: Aufspüren von WWW-Seiten

- Sogenannte "Web Crawler" starten mit einer URL und benutzen Links zu anderen URLs, um sich zu weiteren WWW-Seiten zu hangeln.
- Das ist ein iterativer Prozess: für jede WWW-Seite werden alle Links zu weiteren WWW-Seiten weiterverfolgt.
- Um nicht dieselben Seiten aufgrund zyklischer Links mehrfach abzuarbeiten, wird die URL jeder besichtigten Seite abgespeichert.
- Eine Seite wird nur dann abgearbeitet, wenn die zugehörige URL nicht in der abgespeicherten Menge enthalten ist.



- Beim Hashing werden die Datensätze in einem Array T mit Indizes $0, \dots, m-1$, der sogenannten *Hash-Tabelle*, gespeichert.

Gedankenspiel:

- Wenn wir nun wüssten, dass als Schlüssel nur die Zahlen $0, \dots, m-1$ vorkämen, so könnten wir die Schlüssel direkt als Array-Indizes verwenden.
- Suchen, Einfügen und Löschen gingen dann in $O(1)$.
- Typischerweise ist jedoch die Anzahl der auftretenden Schlüssel sehr viel kleiner als die Anzahl der möglichen Schlüssel.
- Eine Tabelle in der Größenordnung aller möglichen Schlüssel wäre dann eine riesige Speicherplatzverschwendung.



Hash-Funktionen



Idee:

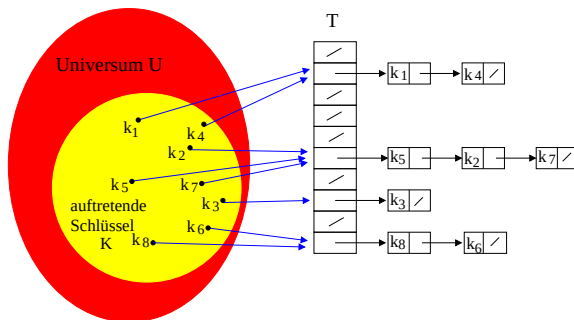
- Wir legen uns eine Tabelle an, deren Größe in etwa proportional zur Anzahl der zu speichernden Datensätze ist.
- Bei n zu speichernden Datensätzen wäre $n \leq m \leq 2n$ zum Beispiel eine sinnvolle Wahl.
- Wir müssen nun für jeden Datensatz einen Array-Index berechnen.
- Das heißt:
Ein Element mit Schlüssel k wird nicht an Position k gespeichert, sondern an Position $h(k)$ mit einer geeigneten Abbildung h .
- Sei U die Menge der möglichen Schlüssel (das Universum). Eine Abbildung $h : U \mapsto \{0, 1, \dots, m-1\}$ nennt man **Hash-Funktion**.
- Für alle $k \in U$ liefert $h(k)$ eine zulässige Position in der Hash-Tabelle T .
- **Sprechweisen:** " k wird nach Position $h(k)$ gehasht" oder " $h(k)$ ist der Hash-Wert von k " oder " $h(k)$ ist die **Hash-Adresse** von k ."



Kollisionsauflösung durch Chaining



Idee: Alle Elemente mit demselben Hash-Wert werden in einer einfach verketteten Liste gehalten.



Position j der Hash-Tabelle T enthält einen Zeiger auf eine Liste aller Elemente mit Hash-Wert j .



Kollisionen



- Nehmen wir also an, dass beim Einfügen ein Datensatz an seiner Hash-Adresse $h(k)$ abgespeichert wird.
- Wenn man das Element dann in der Hash-Tabelle wiederfinden möchte, so berechnet man nach Vorschrift der Hash-Funktion die Hash-Adresse und findet es dann (idealerweise) in $O(1)$ wieder.
- **Problem:** Leider ist es im Allgemeinen nicht ganz so einfach: Wenn das Universum der möglichen Schlüssel mehr Elemente enthält als die Hash-Tabelle, kann die Hash-Funktion nach dem Schubfachprinzip **nicht injektiv** sein.
- Dann gibt es also zwei verschiedene Schlüssel k_1 und k_2 , so dass $h(k_1) = h(k_2)$. Das bezeichnet man beim Hashing als **Kollision**.
- Hashing muss daher um eine Kollisionsbehandlung erweitert werden.



Hashing mit Chaining



Einfügen des Datensatzes x mit Schlüssel $key[x]$:

- Füge x am Kopf der Liste $T[h(key[x])]$ ein.
- Worst-Case-Laufzeit ist $O(1)$.
- Annahme: einzufügendes Element noch nicht in der Liste enthalten.
- Überprüfung dieser Annahme erfordert zusätzliches Suchen.

Suchen eines Datensatzes mit Schlüssel k :

- Suche Element mit Schlüssel k in Liste $T[h(k)]$.
- Die Worst-Case-Laufzeit ist proportional zur Länge der Liste an Position $h(k)$.

Löschen eines Datensatzes mit Schlüssel k :

- Lösche Element mit Schlüssel k aus Liste $T[h(k)]$.
- Auch hier ist die Worst-Case-Laufzeit proportional zur Länge der Liste an Position $h(k)$.



Analyse von Hashing mit Chaining



- Das Worst-Case-Verhalten von Hashing mit Chaining tritt dann auf, wenn alle n Elemente auf den gleichen Eintrag der Hash-Tabelle abgebildet werden.
- Dann hat das Suchen die gleiche Laufzeit wie bei einer einfach verketteten Liste, also $\Theta(n)$.
- Wir wollen nun das Durchschnittsverhalten von Hashing analysieren.
- Unsere Analyse benutzt den Begriff des **Lastfaktors (load factor)** $\alpha := \frac{n}{m}$, wobei
 - n : Anzahl Elemente in der Tabelle
 - m : Anzahl Positionen in der Tabelle
= Anzahl (möglicherweise leerer) verketteter Listen
- Bemerkung: $\alpha < 1$, $= 1$ oder > 1 ist möglich.
- Der Lastfaktor entspricht der **mittleren Anzahl Elemente pro Liste**.
- Der durchschnittliche Aufwand hängt davon ab, wie gleichmäßig h die Schlüssel verteilt.



Durchschnittlicher Aufwand bei erfolgloser Suche



Theorem

In einer Hash-Tabelle mit Kollisionsauflösung durch Chaining beträgt der Erwartungswert der Laufzeit einer erfolglosen Suche $\Theta(1 + \alpha)$.

Beweis:

- Ein noch nicht enthaltener Schlüssel wird mit gleicher Wahrscheinlichkeit in eine der m Listen gehasht.
- Erfolgreiche Suche nach Schlüssel k erfordert vollständiges Durchsuchen der Liste $T[h(k)]$.
- Diese besitzt eine erwartete Länge von $E[n_{h(k)}] = \alpha$.
- Somit beträgt die erwartete Anzahl bei einer erfolglosen Suche zu prüfender Elemente α .
- Zusammen mit der Zeit für die Auswertung der Hash-Funktion ergibt sich $\Theta(1 + \alpha)$. $\square$



Durchschnittlicher Aufwand von Hashing mit Chaining



Annahmen für die weitere Analyse:

- (1) Die Auswertung der Hashfunktion h ist in $O(1)$ Zeit möglich.
- (2) Jedes Element wird mit gleicher Wahrscheinlichkeit in eine der m Positionen von T gehasht (**gleichmäßiges Hashen**).
- (3) Die Wahrscheinlichkeit, dass wir ein bestimmtes Element suchen, ist für alle Elemente gleich, die bereits in der Datenstruktur enthalten sind.

Bezeichne n_j die Länge der Liste $T[j]$, $j = 0, \dots, m-1$.

Dann gilt $n = n_0 + n_1 + \dots + n_{m-1}$.

Wegen (2) ist der Erwartungswert von n_j gegeben durch $E[n_j] = \frac{n}{m} = \alpha$.

Zwei Fälle sind zu unterscheiden:

erfolglose Suche und **erfolgreiche Suche**.



Durchschnittlicher Aufwand bei erfolgreicher Suche



Theorem

In einer Hash-Tabelle mit Kollisionsauflösung durch Chaining beträgt der Erwartungswert der Laufzeit einer erfolgreichen Suche $2 + \frac{\alpha}{2} - \frac{\alpha}{2n} = \Theta(1 + \alpha)$.

Beweis:

- Wir benutzen Annahme (3): das gesuchte Element x ist mit gleicher Wahrscheinlichkeit eines der n in der Tabelle enthaltenen Elemente.
- Die Anzahl der bei Suche nach x zu überprüfenden Elemente ist eins mehr als die Anzahl Vorgänger von x in dessen Liste.
- Diese Elemente wurden nach x eingefügt (da stets am Kopf der Liste eingefügt wird).
- Zu bestimmen ist somit der Erwartungswert für die Anzahl der Elemente, welche nach x in dessen Liste eingefügt wurden.
- Sei x_i das i -te in die Tabelle eingefügte Element ($i = 1, \dots, n$) sowie $k_i := \text{key}[x_i]$.



Durchschnittlicher Aufwand bei erfolgreicher Suche



Fortsetzung des Beweises:

- Für alle $1 \leq i, j \leq n$ definieren wir die Indikatorzufallsvariablen

$$X_{i,j} := \begin{cases} 1, & \text{falls } h(k_i) = h(k_j), \\ 0, & \text{sonst.} \end{cases}$$

- Wegen Annahme (2), dem gleichmäßigen Hashen, gilt $\Pr\{h(k_i) = h(k_j)\} = 1/m$, d.h. $E[X_{i,j}] = 1/m$.
- Sei X die Zufallsgröße, die die Anzahl überprüfter Elemente in einer erfolgreichen Suche angibt. Für deren Erwartungswert erhalten wir

$$E(X) = E\left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{i,j}\right)\right].$$

- Wegen der Linearität des Erwartungswertes und $E[X_{i,j}] = 1/m$ bekommt man

$$E(X) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E(X_{i,j})\right) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m}\right).$$



Gewünschte Eigenschaften von Hash-Funktionen



Was zeichnet eine gute Hash-Funktion aus?

Sie sollte

- leicht und schnell berechenbar sein,
- Kollisionen vermeiden,
- die Datensätze möglichst gleichmäßig verteilt auf die Hash-Tabelle abbilden und
- deterministisch sein (sonst findet man seine Schlüssel ja nicht wieder).

Probleme:

- In der Praxis ist es schwer, die Schlüssel gleichmäßig zu verteilen, da in der Regel die Wahrscheinlichkeitsverteilung der auftretenden Schlüssel nicht im Voraus bekannt ist;
- ferner sind diese möglicherweise nicht unabhängig.



Durchschnittlicher Aufwand bei erfolgreicher Suche



Abschluss des Beweises:

- Der Rest besteht nur noch aus rein algebraischem Umformen:

$$\begin{aligned} E(X) &= 1 + \frac{1}{nm} \sum_{i=1}^n (n-i) \\ &= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) \\ &= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) \\ &= 1 + \frac{n}{m} - \frac{n+1}{2m} = 1 + \frac{n-1}{2m} \\ &= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}. \end{aligned}$$

- Zusammen mit der Zeit für die Auswertung von h ergibt sich als mittlere Gesamtzeit für eine erfolgreiche Suche $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$. $\square$



Kollisionen sind unvermeidbar



Da $|U| \gg m$ kann die Hash-Funktion nicht injektiv sein. Also müssen wir mit Kollisionen rechnen.

Schauen wir uns dazu das sogenannte **Geburtstagsparadoxon** an.

Fragestellung: Wieviele Leute müssen in einem Raum sein, damit die Wahrscheinlichkeit, dass zwei von ihnen am gleichen Tag Geburtstag haben, höher als $1/2$ ist?

(Das Jahr habe dabei 365 Tage, wir berücksichtigen keine Schaltjahre.)

Das lässt sich auf folgende Art berechnen:

- Sei m die Anzahl der möglichen Tage und n die Anzahl der Personen.
- Die Wahrscheinlichkeit q , dass es keine Kollision gibt (also, dass alle Personen an verschiedenen Tagen Geburtstag haben), ist dann

$$q(m, n) = \frac{m \cdot (m-1) \cdot (m-2) \cdot (m-3) \cdots (m-n+1)}{m \cdot m \cdot m \cdot m \cdots m} = \frac{m!}{(m-n)! \cdot m^n}$$

Dann gilt

$$q(365, 23) \approx 0,4927 < \frac{1}{2}.$$



Kollisionen und das Geburtstagsparadoxon



- Es genügen also schon 23 Leute, damit die Wahrscheinlichkeit, dass zwei von ihnen am gleichen Tag Geburtstag haben, größer als $1/2$ ist.

Auf Hash-Funktionen übertragen bedeutet dies folgendes:

- Haben wir ein Universum von 23 Schlüsseln, eine Hash-Tabelle mit 365 Einträgen und eine gleichmäßig verteilende Hash-Funktion, so ist die Wahrscheinlichkeit, dass zwei Schlüssel auf den gleichen Eintrag in der Hash-Tabelle abgebildet werden, größer als $1/2$.
- Gleichzeitig haben wir nur eine **Auslastung** der Hash-Tabelle von nur $\frac{23}{365} \approx 6,301\%$.



Die Divisionsmethode



- Schlüssel k wird auf eine von m Positionen abgebildet durch Restbildung bezüglich Division durch m , d.h.

$$h(k) := k \bmod m.$$

- Beispiel:** Sei $m = 37$ und $k = 224$. Dann ist $h(k) = 2$.
- Frage:** Wie sollte m gewählt werden?
- Beobachtung:** Bestimmte Werte von m sind zu vermeiden.
- Zweierpotenzen sind ungünstig. Bei $m = 2^p$ hängt $h(k)$ nur von den p niedrigstwertigen Bits von k ab. (Der Hash-Wert sollte von allen Bits abhängen, es sei denn alle p -Bitmuster sind gleich wahrscheinlich.)
- Zehnerpotenzen sind ebenfalls ungünstig: Vom Mensch geschaffene Daten sind oft im Dezimalsystem erzeugt.
- Repräsentiert k eine Zeichenkette, die als Zahl zur Basis 2^p dargestellt ist, so ist $m = 2^p - 1$ insofern schlecht, als dann der Hash-Wert invariant ist unter Permutationen der Zeichen. (Nachweis: Übungsaufgabe!)
- Gute Wahl:** wähle m als **Primzahl**, die nicht allzu nahe an einer Potenz des gewählten Zahlensystems liegt, aus dem die Schlüssel stammen.



Schlüssel als natürliche Zahlen



- Wir nehmen im Folgenden an, dass die Schlüsselwerte natürliche Zahlen sind.
- Anderswertige Schlüssel müssen zuvor nach $\mathbb{N}$ abgebildet werden.
- Beispiel:** Abbildung von **ASCII-Zeichenketten** auf natürliche Zahlen in einer geeigneten Basis.

- Betrachte die Zeichenkette CLRS. Die zugehörigen ASCII-Codes lauten C=67, L=76, R=82, S=83.
- ASCII-Codes liegen zwischen 0 und 127. Als **Basis** bietet sich somit 128 an.
- Interpretiere daher die Zeichenkette CLRS als die Zahl

$$(67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0) = 141764947.$$

- Achtung:** Zahlen können dann sehr groß werden.



Die Multiplikationsmethode



- Sei $0 < a < 1$ eine beliebige Konstante.
- Dann benutzt man bei der **Multiplikationsmethode** folgende Hash-Funktion:

$$h : U \mapsto \{0, \dots, m-1\}, \quad h(k) := \lfloor m(ka - \lfloor ka \rfloor) \rfloor.$$

- Erläuterung:** Es wird also k mit einer Konstanten a zwischen 0 und 1 multipliziert, die Vorkomastellen werden abgeschnitten, das Ergebnis wird mit m multipliziert (der dabei entstehende Wert q ist eine reelle Zahl und es gilt $0 \leq q < m$). Dann wird abgerundet und wir erhalten einen ganzzahligen Wert q' mit $0 \leq q' \leq m-1$.
- Bei diesem Verfahren ist die Wahl von m nicht so entscheidend wie bei der Divisionsmethode.
- Nach **D.E. Knuth** ist dabei eine Wahl von

$$a := \frac{\sqrt{5} - 1}{2} \approx 0.6180339887$$

besonders geeignet. (a ist Kehrwert des goldenen Schnitts)



Hashfunktionen und Kryptographie



- Neben den eingeführten Hashfunktionen (Divisions- und Multiplikationsmethode) gibt es noch weitere gebräuchliche Hashfunktionen, auf die wir hier aber nicht näher eingehen wollen.
- Primäres Entwurfsziel für die Anwendung "Suchen in großen Datensätzen" war das Vermeiden von Kollisionen.
- Hashfunktionen haben jedoch auch große Bedeutung im Bereich von **kryptographischen Anwendungen**.
- Beispiele:
 - Test, ob ein Text unversehrt beim Empfänger angekommen ist.
 - Test, ob eine Datei (z.B. Passwortdatei oder Programm) unverändert ist.
 - Digitale Signatur (Nachweis des Absenders).



Offene Adressierung



Anstelle einer Kollisionsbehandlung mit Listen kann man auch mit sogenannter **offener Adressierung** arbeiten.

Idee:

- Alle Datensätze werden direkt in der Hash-Tabelle gespeichert.
- An jeder Position steht entweder ein Datensatz oder null.
- Tritt beim Einfügen eine Kollision ein, so wird versucht, innerhalb der Tabelle einen freien Platz zu finden.
- Dazu führt man eine **erweiterte Hashfunktion** $h(k, i)$ ein, die angibt, an welche Position der Datensatz gehasht werden soll, wenn bereits i Versuche durchgeführt worden sind.
- Den Test, ob Position $h(k)$ frei, mit Schlüssel k oder einem anderen Schlüssel belegt ist, nennen wir **Sondierung (probe)**.
- Damit falls notwendig alle Positionen sondiert werden und jede Position höchstens einmal sondiert wird, sollte die Folge der Sondierungen eine Permutation der Zahlen $\{0, 1, \dots, m-1\}$ sein.



Kryptographische Hashfunktionen



- Für kryptographische Hashfunktionen braucht man als Eigenschaft, dass sie kollisionsresistent sind.
- Eine Hashfunktion h heißt **schwach kollisionsresistent**, wenn es zu vorgegebenen Schlüssel k schwer ist, einen anderen Schlüssel k' zu finden, für den $h(k) = h(k')$ gilt.
- Eine Hashfunktion h heißt **stark kollisionsresistent**, wenn es praktisch unmöglich ist, irgendeine Kollision zu finden.
- Es ist nicht bekannt, ob es solche Funktionen überhaupt gibt.
- In der Praxis verwendet man Funktionen, bei denen Kollisionsresistenz bisher nicht widerlegt wurde.
- SHA-1 ist eine der gebräuchlichsten kryptographischen Hashfunktionen.
- Näheres dazu in Vorlesungen zu Kryptographie und Datensicherheit.



Offene Adressierung



- Die Hash-Funktion ist somit eine Abbildung

$$h : U \times \{0, 1, \dots, m-1\} \mapsto \{0, 1, \dots, m-1\}.$$

- Die Forderung, dass für jeden Schlüssel die Folge der generierten Positionen eine Permutation von $\{0, 1, \dots, m-1\}$ ist, bedeutet: die Sondierungsfolge $\{h(k, 0), h(k, 1), \dots, h(k, m-1)\}$ ist eine Permutation von $\{0, 1, \dots, m-1\}$.

Suchen nach Schlüssel k :

- Prüfe, ob k an Position $h(k, 0)$ steht.
- Enthält Position $h(k, 0)$ den Schlüssel k , ist die Suche erfolgreich; enthält Position $h(k, 0)$ den Wert null, ist sie erfolglos.
- Dritte Möglichkeit: Position $h(k)$ enthält anderen Schlüssel $k' \neq k$. In diesem Fall werden, abhängig von k und der aktuellen Sondierung, solange weitere mögliche Positionen für k durchlaufen, bis an einer dieser entweder null oder der Schlüssel k vorgefunden wird.



Einfügen bei offener Adressierung



Vorgehen beim Einfügen (von Datensatz d mit Schlüssel k):

Annahme: es gibt noch mindestens einen freien Platz in der Tabelle.

```

INSERT ( $d, k$ ){
  falls Tabelle voll ist, stop;
  i=0;
  WHILE nicht fertig DO
    IF  $h(k, i) == \text{null}$  THEN
      speichere Datensatz an Stelle  $h(k, i)$ ; stop;
    ELSE
       $i \leftarrow i + 1$ ;
  }

```

Problem: Was machen wir, wenn die Tabelle voll wird?

Mögliche Lösung:

- Wir legen eine neue, größere Tabelle an (z.B. doppelt so groß).
- Danach iterieren wir über alle Elemente der alten Tabelle und fügen sie sukzessive in die neue Tabelle ein.



Löschen bei offener Adressierung



Mögliche Lösung des Problems:

- Verwendung eines Sonderzeichens DELETED neben null.
- Beim Suchen werden Positionen mit Symbol DELETED behandelt als enthielten diese vom gesuchten Schlüssel verschiedene Schlüssel.
- Beim Einfügen werden solche Positionen als frei erkannt und können erneut belegt werden.
- **Nachteil:** Suchzeit hängt nun nicht mehr alleine vom Lastfaktor ab, sondern kann sich graduell mit der Anzahl der Löschoperationen verschlechtern.
- Wenn man unbedingt Löschen unterstützen will, benutzt man daher eher Kollisionsbehandlung mit Listen.



Löschen bei offener Adressierung



Löschen von Elementen ist bei offener Adressierung **ein Problem**:

Es genügt **nicht**, die Position des zu löschenden Datensatzes mit null zu überschreiben.

Begründung:

- Der zu löschende Datensatz habe Schlüssel k und stehe an Position j .
- Nach Einfügen von k sei ein weiterer Schlüssel k' eingefügt worden, im Verlaufe dessen Einfügens Position j sondiert (und k vorgefunden) wurde.
- Angenommen wir löschen daraufhin Schlüssel k durch Eintragen von null an Position j .
- Suchten wir daraufhin nach Schlüssel k' , so würde vor der k' enthaltenden Position zuerst Position j sondiert, null vorgefunden und die Suche als erfolglos beendet werden.
- Die Antwort wäre falsch, da k' noch in der Tabelle enthalten wäre.



Lineares Sondieren (linear probing)



- Zu einer gegebenen Hash-Funktion $h' : U \mapsto \{0, 1, \dots, m-1\}$ verwendet **lineares Sondieren** die Hash-Funktion

$$h(k, i) = (h'(k) + i) \bmod m, \quad i = 0, 1, \dots, m-1.$$

- Zu gegebenem Schlüssel k wird also zuerst $T[h'(k)]$ sondiert gefolgt von $T[h'(k) + 1], \dots, T[m-1], T[0], T[1], \dots, T[h'(k) - 1]$.
- Die erste Sondierung bestimmt die gesamte Folge, es gibt genau m verschiedene Sondierungsfolgen.
- Lineares Sondieren ist einfach zu implementieren,
- leidet aber unter der Bildung von sogenannten **primären Clustern**: eine freie Position, welcher i besetzte Positionen vorangehen, wird (bei gleichmäßigem Hashen von h') mit Wahrscheinlichkeit $(i+1)/m$ besetzt.
- So neigen lange Folgen besetzter Positionen dazu, noch länger zu werden, womit sich die Suchzeit erhöht.



Quadratisches Sondieren (quadratic probing)



- Beim **quadratischen Sondieren** lautet zu gegebener Hashfunktion h' und Konstanten $c_1, c_2 \neq 0$ die Hash-Funktion

$$h(k, i) = (h'(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m, \quad i = 0, 1, \dots, m-1.$$
- Wieder wird $T[h'(k)]$ zuerst sondiert; die Inkremente zu den danach sondierten Positionen hängen quadratisch von i ab.
- Quadratisches Sondieren funktioniert deutlich besser als lineares Sondieren.
- Es ist jedoch erforderlich, c_1 , c_2 und m geeignet zu wählen, um das Feld möglichst gut auszunutzen.
- Für zwei Schlüssel k_1, k_2 mit $h(k_1, 0) = h(k_2, 0)$ gilt $h(k_1, i) = h(k_2, i)$ für alle i ; diese Eigenschaft führt zur Bildung von sogenannten **sekundären Clustern**.
- Wieder bestimmt die erste Sondierposition alle weiteren, es gibt also auch hier genau m verschiedene Sondierfolgen.



Analyse von offener Adressierung



Annahmen:

- Gleiche Annahmen wie bei Kollisionsbehandlung mit Listen (gleichmäßiges Hashen und bei erfolgreichen Suchen wird jeder Schlüssel mit gleicher Wahrscheinlichkeit gesucht)
- Zusätzliche Annahmen:
 - Tabelle nie vollständig belegt, d.h. Lastfaktor $\alpha = n/m < 1$.
 - Keine Löschoperationen

Theorem

Bei einer Hash-Tabelle mit offener Adressierung und Lastfaktor $\alpha = n/m < 1$ ist die erwartete Anzahl Sondierungen eine von α abhängige Konstante. Also geht Suchen in solchen Hash-Tabellen im Mittel in $O(1)$, solange sich der Lastfaktor nicht ändert.

Bemerkung:

- Die genaue Konstante hängt vom gewählten Sondierungsverfahren ab.
- Wir verzichten hier auf eine genaue Berechnung und den Beweis.



Doppeltes Hashen



- Beim *doppelten Hashen* werden zwei Hashfunktionen h_1 und h_2 benutzt: h_1 liefert die erste Sondierung, h_2 bestimmt die weiteren.
- Die erweiterte Hashfunktion lautet

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m.$$

- $h_2(k)$ muss teilerfremd zu m sein, damit eine Sondierungsfolge eine Permutation der Tabellenpositionen bildet.
- Mögliche Wahl: m ist eine Primzahl mit $1 < h_2(k) < m$.
- Verbesserung gegenüber linearer oder quadratischer Sondierung:
Es gibt $\Theta(m^2)$ verschiedene Sondierungsfolgen anstelle von $\Theta(m)$,
(denn jedes Paar $(h_1(k), h_2(k))$ liefert andere Folge).